

04/6/2026

INFORME_TP1_JUNIT

Analista en sistemas - Ingeniería de Software II -
prof. Caseres Marina

Alumno: Agustin Nicolas Carsaniga

[https://github.com/ElPelucaDLAP/
TP1_JUnit_AgustinCarsaniga](https://github.com/ElPelucaDLAP/TP1_JUnit_AgustinCarsaniga)

1.Agenda Personal descripción:	3
2.Testeo:	4
2.1Contacto:	4
Funcionalidad - Constructor de la clase Contacto:	4
Test 01 - Nombre nulo:	4
Test 02 - Nombre vacío:	5
Test 03 - Mail sin "@":	5
Test 04 - Test mail null:	5
Test 05 - Mail vacío:	5
Funcionalidad - Método equals sobrescrito:	6
Test 01 - Contactos con nombres distintos:	6
Test 02 - Contactos con nombres idénticos:	6
Test 03 - Comparación con null:	6
Test 04 - Comparación con un objeto de otro tipo:	7
Test 05 - Comparación con objeto base con parametro null:	7
Resultado de los test:	7
2.1Reunión:	8
Funcionalidad - Constructor de la clase Reunión:	8
Test 01 - Hora de inicio nula:	8
Test 02 - Hora de fin nula:	8
Test 03 - Hora de inicio posterior a la de fin:	8
Test 04 - Hora de inicio igual a la de fin:	9
Funcionalidad - Método añadirParticipante:	9
Test 01 - Adición de un participante válido:	9
Test 02 - Adición de un participante duplicado:	10
Test 03 - Adición de un participante nulo:	10
Funcionalidad - Método existeContacto:	10
Test 01 - Consulta con participante nulo:	11
Test 02 - Consulta de participante inexistente:	11
Test 03 - Consulta de participante existente:	11
Funcionalidad - Método compartimosHorario:	11
Test 01 - Sin conflicto de horario:	12
Test 02 - Conflicto por inicio previo con fin superpuesto:	12
Test 03 - Conflicto por inicio posterior dentro del intervalo:	12
Test 04 - Conflicto por reunión completamente contenida:	12
Resultado de los test:	13
2.3Agenda Personal:	13
Funcionalidad - Método añadirContacto:	13
Test 01 - Contacto nulo:	14
Test 02 - Adición de contactos distintos:	14
Test 03 - Adición de contacto duplicado:	14
Funcionalidad - Método añadirReunion:	15
Test 01 - Reunión nula:	15
Test 02 - Reunión con conflicto de horario:	15
Test 03 - Reunión sin conflicto de horario:	16
Funcionalidad - Método existeConflictoReunion :	16
Test 01 - Sin conflicto de horario:	16
Test 02 - Conflicto por superposición directa:	17
Test 03 - Conflicto por inicio posterior dentro del intervalo:	17
Test 04 - Conflicto por reunión completamente contenida:	17
Test 05 - Parámetro nulo:	17
Funcionalidad - Método existeConflictoContacto:	18
Test 01 - Contacto existente:	18
Test 02 - Parámetro nulo:	18
Resultado de los test:	19
Reflexión final:	19
Repositorio de git:	19

1. Agenda Personal descripción:

El programa seleccionado es una agenda personal destinada a registrar y gestionar reuniones. Está compuesto por tres clases principales: Contacto, Reunión y Agenda Personal.

La clase Contacto representa a las personas registradas en la agenda, con su nombre, teléfono y mail. Su método principal permite identificar cada contacto de manera única, evitando la duplicación de nombres dentro del sistema.

La clase Reunión representa eventos que se desarrollan durante un período de tiempo determinado. Cada reunión posee una lista de contactos participantes, una fecha con horario de inicio y fin, un tema a tratar y un lugar donde se llevará a cabo. Entre sus métodos más importantes se encuentran la posibilidad de agregar participantes, verificar si un contacto ya forma parte de la reunión y detectar conflictos de horario con otras reuniones.

Por último, la clase Agenda Personal actúa como contenedor principal del sistema, almacenando las listas de contactos y reuniones registradas. Sus métodos permiten agregar nuevos contactos, registrar reuniones, buscar conflictos entre reuniones, detectar conflictos relacionados con los contactos, obtener la última reunión registrada y recuperar el último contacto incorporado a la agenda.

2. Testeo:

2.1 Contacto:

Funcionalidad - Constructor de la clase Contacto:

```
public Contacto(String nombre, String telefono, String mail){ 15 usages  elpeluca
    if (nombre == null || nombre.isEmpty()){
        throw new IllegalArgumentException("Falta de Valores necesarios");
    }
    if (mail == null || !mail.contains("@")){
        throw new IllegalArgumentException("Falta de Valores necesarios");
    }
    this.nombre=nombre;
    this.telefono=telefono;
    this.mail=mail;
}
```

Descripción:

El constructor de la clase Contacto incluye validaciones que garantizan el cumplimiento de requisitos mínimos antes de crear una instancia válida. Concretamente, se verifica que el nombre no sea null ni esté vacío, y que el mail no sea null y contenga el carácter "@". Si alguna de estas condiciones no se cumple, el constructor lanza una excepción indicando el motivo del fallo.

Test 01 - Nombre nulo:

```
@Test  @ elpeluca
void testThrowsConstructorNombreNull(){
    assertThrows(IllegalArgumentException.class, ()->{
        new Contacto( nombre: null, telefono: "2494066642", mail: "1@gmail");
    });
}
```

Se intenta construir un objeto Contacto pasando null como nombre. El uso de assertThrows verifica que el constructor reaccione correctamente ante este dato inválido, lanzando una excepción que indica que el tipo del argumento es inválido.

Test 02 - Nombre vacío:

```
@Test  @ elpeluca
void testThrowsConstructorNombreVacio(){
    assertThrows(IllegalArgumentException.class, ()->{
        new Contacto( nombre: "", telefono: "2494066642", mail: "1@gmail");
    });
}
```

Nombre vacío: Se intenta construir un objeto Contacto con una cadena vacía como nombre. El assertThrows confirma que el constructor detecta la ausencia de un nombre real y lanza la excepción correspondiente.

Test 03 - Mail sin "@":

```
@Test  @ elpeluca
void testThrowsConstructorMailSinArroba(){
    assertThrows(IllegalArgumentException.class, ()->{
        new Contacto( nombre: "Roberto", telefono: "2494066642", mail: "1gmail");
    });
}
```

Se intenta construir un Contacto con un mail que no contiene el símbolo "@". El assertThrows verifica que el constructor identifica el formato incorrecto del mail y lanza la excepción apropiada.

Test 04 - Test mail null:

```
@Test  @ elpeluca
void testThrowsConstructorMailNull(){
    assertThrows(IllegalArgumentException.class, ()->{
        new Contacto( nombre: "Roberto", telefono: "2494066642", mail: null);
    });
}
```

Se intenta construir un Contacto con un mail null. El assertThrows confirma que el constructor detecta la ausencia de un "@" y lanza la excepción correspondiente.

Test 05 - Mail vacío:

```
@Test  @ elpeluca
void testThrowsConstructorMailVacio(){
    assertThrows(IllegalArgumentException.class, ()->{
        new Contacto( nombre: "Roberto", telefono: "2494066642", mail: "");
    });
}
```

Se intenta construir un Contacto con un mail vacío. El assertThrows confirma que el constructor detecta la ausencia de un mail y lanza la excepción correspondiente.

Funcionalidad - Método equals sobrescrito:

```
@Override  @ elpeluca
public boolean equals(Object obj) {
    if (obj == null){
        return false;
    }
    if (this.getClass() !=obj.getClass()){
        return false;
    }
    Contacto otroContacto = (Contacto) obj;
    if (this.getNombre() == null){
        return otroContacto.getNombre() == null;
    }
    return this.getNombre().equals(otroContacto.getNombre());
}
```

Descripción:

Se sobrescribe el método equals para comparar dos instancias de Contacto e identificar si representan el mismo contacto, tomando el nombre como criterio de igualdad. La implementación incluye salvaguardas para evitar errores o falsos positivos al comparar con objetos null o de un tipo distinto.

Test 01 - Contactos con nombres distintos:

```
@Test @elpeluca *
void testEqualsIdentificarDiferente () { assertFalse(contactoDePrueba1.equals(contactoDePrueba2)); }
```

Se comparan dos instancias de Contacto cuyos nombres difieren. El assertFalse valida que el método equals reconoce que no son el mismo contacto y retorna false.

Test 02 - Contactos con nombres idénticos:

```
@Test @elpeluca
void testEqualsIdentificarIgual () {
    Contacto mismo = new Contacto( nombre: "Roberto", telefono: "2494066652", mail: "1@gmail");
    assertTrue(contactoDePrueba1.equals(mismo));
}
```

Se comparan dos instancias de Contacto cuyos nombres son exactamente iguales carácter por carácter. El assertTrue valida que el método equals los reconoce como equivalentes y retorna true.

Test 03 - Comparación con null:

```
@Test @elpeluca
void testEqualsDatosNull () { assertFalse(contactoDePrueba1.equals(null)); }
```

Se compara un Contacto válido con null. El assertFalse verifica que el método equals maneja este caso sin lanzar excepciones y retorna false, reconociendo que la comparación no es válida.

Test 04 - Comparación con un objeto de otro tipo:

```
@Test @elpeluca
void testEqualDatosInvalidos () { assertFalse(contactoDePrueba1.equals(reunionDePrueba)); }
```

Comparación con un objeto de otro tipo, se compara un contacto con un objeto de una clase diferente. El assertFalse verifica que el método equals detecta la incompatibilidad de tipos y retorna false sin producir errores.

Test 05 - Comparación con objeto base con parametro null:

```
@Test @elpeluca
void testEqualContactoComparable () {
    contactoDePrueba1.setNombre(null);
    assertFalse(contactoDePrueba1.equals(contactoDePrueba2));
}
```

Comparación con un objeto null se compara con un objeto de tipo null. El assertFalse verifica que el método equals detecta la incompatibilidad de tipos y retorna false sin producir errores.

Resultado de los test:

```
[INFO] Running ContactoTest
Iniciando suite de pruebas de Contacto
*
*
*
*
*
*
*
*
*
*
*
=== Suite finalizada ===
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.092 s -- in ContactoTest
```

Todos los test pasaron.

2.1 Reunión:

Funcionalidad - Constructor de la clase Reunión:

```
public Reunion (LocalDateTime inicioReunion, LocalDateTime finReunion, String temaAtratar, String lugarDeReunion){
    if (inicioReunion == null || finReunion == null) {
        throw new IllegalArgumentException("Las fechas de la reunión no pueden ser nulas");
    }
    if (finReunion.isBefore(inicioReunion) || finReunion.isEqual(inicioReunion)) {
        throw new IllegalArgumentException("La fecha de fin debe ser posterior a la fecha de inicio");
    }
    this.contactosDeReunion = new ArrayList();
    this.inicioReunion=inicioReunion;
    this.finReunion=finReunion;
    this.temaAtratar=temaAtratar;
    this.lugarDeReunion=lugarDeReunion;
}
```

16 usages  elpeluca

Descripción:

El constructor de la clase Reunión incluye validaciones que impiden la creación de instancias inválidas. Concretamente, se verifica que tanto la hora de inicio como la de fin estén asignadas (no sean null), y que la hora de fin sea estrictamente posterior a la de inicio. Si alguna de estas condiciones no se cumple, el constructor lanza una excepción describiendo el motivo del fallo.

Test 01 - Hora de inicio nula:

```
@Test @elpeluca
void testThrowsConstructorInicioReunionNull () {
    assertThrows(IllegalArgumentException.class, ()->{
        new Reunion( inicioReunion: null, LocalDateTime.of( year: 2026, month: 6, dayOfMonth: 20, hour: 14, minute: 0), temaAtrator: "AS", lugarDeReunion: "av 123");
    });
}
```

Se intenta crear una Reunión con null como hora de inicio. El `assertThrows` verifica que el constructor detecta el argumento inválido y lanza la excepción correspondiente.

Test 02 - Hora de fin nula:

```
@Test
@elpeluca
void testThrowsConstructorFinReunionNull () {
    assertThrows(IllegalArgumentException.class, ()->{
        new Reunion(LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 12, minute: 0), finReunion: null, temaAtratar: "AS", lugarDeReunion: "av 123");
    });
}
```

Se intenta crear una Reunión con null como hora de fin. El `assertThrows` verifica que el constructor detecta el argumento inválido y lanza la excepción correspondiente

Test 03 - Hora de inicio posterior a la de fin:

```
@Test @elpeluca *
void testThrowsConstructorFinDeReunionMenorInicio () {
    assertThrows(IllegalArgumentException.class, ()->{
        new Reunion(LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 12, minute: 0), LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 11, minute: 0),
            temaAtratar: "AS", lugarDeReunion: "av 123");
    });
}
```

Se intenta crear una Reunión donde la hora de inicio es mayor a la de fin. El `assertThrows` confirma que el constructor identifica esta condición como inválida y lanza la excepción apropiada.

Test 04 - Hora de inicio igual a la de fin:

```
@Test @elpeluca *
void testThrowsConstructorFinDeReunionIgualInicio () {
    assertThrows(IllegalArgumentException.class, ()->{
        new Reunion(LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 12, minute: 0), LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 12, minute: 0),
            temaAtratar: "AS", lugarDeReunion: "av 123");
    });
}
```

Se intenta crear una Reunión donde ambos horarios coinciden exactamente. El `assertThrows` confirma que el constructor rechaza este caso, ya que una reunión de duración cero no es válida, y lanza la excepción correspondiente.

Funcionalidad - Método `añadirParticipante`:

```
public void añadirParticipante(Contacto nuevoParticipante){ 6 usages @elpeluca *
    if (nuevoParticipante==null){
        return;
    }
    if (!yaExisteContacto(nuevoParticipante)){
        this.contactosDeReunion.add(nuevoParticipante);
    }
}
```

Descripción:

Este método permite incorporar contactos como participantes a una reunión. Antes de agregar un nuevo participante, verifica que el objeto recibido no sea null y que no exista ya un participante equivalente en la lista, evitando duplicados y errores en tiempo de ejecución.

Test 01 - Adición de un participante válido:

```
@Test @elpeluca
void testAñadirParticipantes (){
    reunion1.añadirParticipante(participante1);
    Contacto ultimoAgregado = reunion1.obtenerUltimoContactoRegistrado();
    assertEquals(participante1, ultimoAgregado);
}
```


Se añade un participante a la reunión y luego se recupera el último elemento de la lista de participantes. El assertEquals compara ambos para confirmar que el participante fue incorporado correctamente.

Test 02 - Adición de un participante duplicado:

```
@Test  👤 elpeluca
void testAnadirParticipantesIguales (){
    reunion1.anadirParticipante(participante2);
    reunion1.anadirParticipante(participante1);
    Contacto ultimoAgregado = reunion1.obtenerUltimoContactoRegistrado();
    assertEquals(participante2, ultimoAgregado);
}
```

Se añaden dos participantes con el mismo nombre a la reunión. Luego se verifica que el último participante de la lista sigue siendo el primero que se añadió, comprobando mediante assertEquals que el método rechaza la incorporación de duplicados.

Test 03 - Adición de un participante nulo:

```
@Test  👤 elpeluca
void testAnadirParticipantesValoresNull (){
    reunion1.anadirParticipante( nuevoParticipante: null);
    assertEquals( expected: 0, reunion1.getCantidadParticipantes());
}
```

Se intenta añadir null como participante. El assertEquals verifica que el tamaño de la lista de participantes permanece en 0, confirmando que el método ignoró el intento de incorporación.

Funcionalidad - Método existeContacto:

```
public boolean yaExisteContacto (Contacto nuevoParticipante) { 3 usages  👤 elpeluca
    if (nuevoParticipante==null){
        return false;
    }
    boolean existe = false;
    int i = 0;
    while (i<this.contactosDeReunion.size() && !existe) {
        if(this.contactosDeReunion.get(i).equals(nuevoParticipante)){
            existe=true;
        }else {
            i++;
        }
    }
    return existe;
}
```

Descripción:

Este método recorre la lista de participantes de la reunión para determinar si un contacto dado ya se encuentra registrado. También contempla el caso en que el contacto recibido sea null, retornando false sin producir errores.

Test 01 - Consulta con participante nulo:

```
@Test new *
void testYaExisteContactoNull () {
    assertFalse(reunion1.yaExisteContacto( nuevoParticipante: null));
}
```

Se consulta la existencia de null en la lista. El assertFalse verifica que el método detecta el argumento inválido y retorna false correctamente.

Test 02 - Consulta de participante inexistente:

```
@Test @ elpeluca *
void testYaExisteContactoNuevo () {
    reunion1.anadirParticipante(participante1);
    assertFalse(reunion1.yaExisteContacto(participante3));
}
```

Se añade un participante a la reunión y luego se consulta por uno diferente. El assertFalse verifica que el método reconoce que no son equivalentes y retorna false.

Test 03 - Consulta de participante existente:

```
@Test @ elpeluca *
void testYaExisteContactoIdentifico () {
    reunion1.yaExisteContacto(participante1);
    assertTrue(reunion1.yaExisteContacto(participante2));
}
```

Se añade un participante a la reunión y luego se consulta por uno con el mismo nombre. El assertTrue verifica que el método los identifica como iguales y retorna true.

Funcionalidad - Método compartimosHorario:

```
public boolean compartimosHorario (Reunion reunionNueva){ 5 usages @ elpeluca
    if (reunionNueva == null) {
        return false;
    }
    return this.inicioReunion.isBefore(reunionNueva.getFinReunion())
        && reunionNueva.getInicioReunion().isBefore(this.finReunion);
}
```

Descripción:

Este método determina si una reunión nueva presenta un conflicto de horario con la reunión actual, comparando sus intervalos de inicio y fin. También valida que la reunión recibida por parámetro no sea null antes de realizar la comparación.

Test 01 - Sin conflicto de horario:

```
@Test @elpeluca *
void testCompartimosHorarioNoSePisan () {
    Reunion reunion2 = new Reunion(LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 14, minute: 0),
    LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 16, minute: 0), temaAtratar: "AS", lugarDeReunion: "av 123");
    assertFalse(reunion1.compartimosHorario(reunion2));
}
```

Se crea una reunión cuyo horario no se superpone con el de la reunión de comparación. El `assertFalse` confirma que el método no detecta conflicto y retorna false.

Test 02 - Conflicto por inicio previo con fin superpuesto:

```
@Test @elpeluca *
void testCompartimosHorarioEsAnterior () {
    Reunion reunion5 = new Reunion(LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 10, minute: 30),
    LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 12, minute: 30), temaAtratar: "AS", lugarDeReunion: "av 123");
    assertTrue(reunion1.compartimosHorario(reunion5));
}
```

Se crea una reunión que comienza antes que la de comparación pero finaliza después de que esta haya comenzado, generando una superposición. El `assertTrue` confirma que el método detecta el conflicto y retorna true.

Test 03 - Conflicto por inicio posterior dentro del intervalo:

```
@Test @elpeluca *
void testCompartimosHorarioArrancaAntes () {
    Reunion reunion3 = new Reunion(LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 13, minute: 30),
    LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 15, minute: 0), temaAtratar: "AS", lugarDeReunion: "av 123");
    assertTrue(reunion1.compartimosHorario(reunion3));
}
```

Se crea una reunión cuyo inicio ocurre dentro del intervalo de la reunión de comparación. El `assertTrue` confirma que el método detecta la superposición y retorna true.

Test 04 - Conflicto por reunión completamente contenida:

```
@Test @elpeluca *
void testCompartimosHorarioContenidaDentroDeOtra () {
    Reunion reunion4 = new Reunion(LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 12, minute: 30),
    LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 13, minute: 30), temaAtratar: "AS", lugarDeReunion: "av 123");
    assertTrue(reunion1.compartimosHorario(reunion4));
}
```

Se crea una reunión cuyo inicio es posterior y cuyo fin es anterior al de la reunión de comparación, quedando completamente dentro de su intervalo. El `assertTrue` confirma que el método detecta el conflicto y retorna true.

Resultado de los test:

```
[INFO] Running ReunionTest
Finalizando prueba
    Iniciando suite de pruebas de Reunion
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
=== Suite finalizada ===
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.061 s -- in ReunionTest
[INFO] Running AppendResponseTest
```

Todos los test pasaron.

2.3 Agenda Personal:

Funcionalidad - Método añadirContacto:

```
public void anadirContacto (Contacto nuevoContacto) { 6 usages  👤 elpeluca *
    if(nuevoContacto == null){
        throw new IllegalArgumentException("El contacto no puede ser null");
    }

    if (existeConflictoContacto(nuevoContacto)){
        throw new IllegalStateException("Conflicto contacto identico");
    }else {
        this.contactos.add(nuevoContacto);
    }
}
```

Descripción:

Este método incorpora un nuevo contacto a la lista de la agenda. Antes de agregarlo, verifica que el contacto no sea null y que no se encuentre ya registrado, evitando tanto errores en tiempo de ejecución como duplicados en la lista.

Test 01 - Contacto nulo:

```
@Test @ elpeluca *
void testAnadirConctactoNull (){
    assertThrows(IllegalArgumentException.class, () ->
        agendaPersonal.anadirContacto( nuevoContacto: null)
    );
}
```

Se intenta añadir null como contacto. El `assertThrows` verifica que el método detecta el argumento inválido y lanza la excepción correspondiente.

Test 02 - Adición de contactos distintos:

```
@Test @ elpeluca
void testAnadirContactoDiferente (){
    agendaPersonal.anadirContacto(participante1);
    agendaPersonal.anadirContacto(participante3);
    assertEquals(participante3, agendaPersonal.obtenerUltimoContactoRegistrado());
}
```

Se añaden dos contactos diferentes a la agenda. El `assertEquals` verifica que el último contacto registrado coincide con el segundo que se añadió, confirmando que ambas incorporaciones se realizaron correctamente.

Test 03 - Adición de contacto duplicado:

```
@Test @ elpeluca *
void testAnadirContactoIgual (){
    agendaPersonal.anadirContacto(participante1);
    assertThrows(IllegalStateException.class, () ->
        agendaPersonal.anadirContacto(participante2)
    );
    assertEquals(participante1, agendaPersonal.obtenerUltimoContactoRegistrado());
}
```

Se intenta añadir dos contactos con el mismo nombre. El `assertThrows` verifica que el método lanza una excepción al detectar el duplicado, y el `assertEquals` confirma que el último contacto registrado sigue siendo el primero que se añadió, demostrando que el segundo fue rechazado.

Funcionalidad - Método añadirReunion:

```
public void anadirReunion (Reunion nuevaReunion){ 7 usages  @elpeluca *
    if (nuevaReunion == null) {
        throw new IllegalArgumentException("La reunión no puede ser null");
    }
    if (existeConflictoReunion(nuevaReunion)){
        throw new IllegalStateException("Conflicto de horario con otra reunión");
    }else{
        this.reunionesAgendadas.add(nuevaReunion);
    }
}
```

Descripción:

Este método registra nuevas reuniones en la agenda. Antes de incorporar una reunión, verifica que no sea null y que su horario no entre en conflicto con alguna reunión ya agendada, garantizando la integridad del calendario.

Test 01 - Reunión nula:

```
@Test @elpeluca *
void testAnadirReunionNull (){
    assertThrows(IllegalArgumentException.class, () ->
        agendaPersonal.anadirReunion( nuevaReunion: null)
    );
}
```

Se intenta añadir null como reunión. El assertThrows verifica que el método detecta el argumento inválido y lanza la excepción correspondiente.

Test 02 - Reunión con conflicto de horario:

```
@Test @elpeluca *
void testAnadirReunionIgual (){
    agendaPersonal.anadirReunion(reunion1);
    Reunion reunion2 = new Reunion(
        LocalDateTime.of( year: 2026, month: 6, dayOfMonth: 20, hour: 12, minute: 0),
        LocalDateTime.of( year: 2026, month: 6, dayOfMonth: 20, hour: 14, minute: 0), temaAtratar: "AS", lugarDeReunion: "av 123"
    );
    assertThrows(IllegalStateException.class, () ->
        agendaPersonal.anadirReunion(reunion2)
    );
    assertEquals(reunion1, agendaPersonal.obtenerUltimoReunionRegistrado());
}
```

Se intenta añadir una reunión cuyo horario se superpone con una ya registrada. El assertThrows verifica que el método lanza una excepción ante el conflicto detectado, y el assertEquals confirma que la última reunión registrada sigue siendo la original, comprobando que la nueva fue rechazada.

Test 03 - Reunión sin conflicto de horario:

```
@Test
public void testAnadirReunionDiferente () {
    agendaPersonal.anadirReunion(reunion1);
    Reunion reunion3 = new Reunion(LocalDate.of(2026, 6, 20, 16, 0),
        LocalDate.of(2026, 6, 20, 18, 0), "AS", "av 123");
    agendaPersonal.anadirReunion(reunion3);
    assertEquals(reunion3, agendaPersonal.obtenerUltimoReunionRegistrado());
}
```

Se añaden dos reuniones con horarios compatibles. El assertEquals verifica que la última reunión registrada corresponde a la segunda añadida, confirmando que fue incorporada exitosamente al no presentar conflictos.

Funcionalidad - Método existeConflictoReunion :

```
public boolean existeConflictoReunion (Reunion nuevaReunion) {
    if (nuevaReunion == null) {
        return true;
    }
    int i = 0;
    boolean existe = false;
    while (i < this.reunionesAgendadas.size() && !existe) {
        if (this.reunionesAgendadas.get(i).compartimosHorario(nuevaReunion)) {
            existe = true;
        } else {
            i++;
        }
    }
    return existe;
}
```

Descripción:

Este método recorre la lista de reuniones de la agenda y consulta, una por una, si alguna presenta conflicto de horario con la reunión recibida por parámetro. Incluye validación para rechazar reuniones null sin producir errores.

Test 01 - Sin conflicto de horario:

```
@Test
public void testCompartimosHorarioNoSePisan () {
    agendaPersonal.anadirReunion(reunion1);
    Reunion reunion2 = new Reunion(LocalDate.of(2026, 6, 20, 14, 0),
        LocalDate.of(2026, 6, 20, 16, 0), "AS", "av 123");
    assertFalse(agendaPersonal.existeConflictoReunion(reunion2));
}
```

Se registra una reunión en la agenda y se crea otra de forma local con un horario no superpuesto. El assertFalse verifica que al invocar existeConflicto desde la agenda con la reunión local, el método retorna false correctamente.

Test 02 - Conflicto por superposición directa:

```
@Test @elpeluca *
void testCompartimosHorarioEsAnterior () {
    agendaPersonal.anadirReunion(reunion1);
    Reunion reunion5 = new Reunion(LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 10, minute: 30),
        LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 12, minute: 30), temaAtratar: "AS", lugarDeReunion: "av 123");
    assertTrue(agendaPersonal.existeConflictoReunion(reunion5));
}
```

Se registra una reunión en la agenda y se crea otra de forma local cuyo horario se superpone con ella. El assertTrue verifica que existeConflicto detecta la superposición y retorna true.

Test 03 - Conflicto por inicio posterior dentro del intervalo:

```
@Test @elpeluca *
void testCompartimosHorarioArrancaAntes () {
    agendaPersonal.anadirReunion(reunion1);
    Reunion reunion3 = new Reunion(LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 13, minute: 30),
        LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 15, minute: 0), temaAtratar: "AS", lugarDeReunion: "av 123");
    assertTrue(agendaPersonal.existeConflictoReunion(reunion3));
}
```

Se registra una reunión en la agenda y se crea otra de forma local que comienza después pero dentro del intervalo de la primera. El assertTrue verifica que existeConflicto identifica el solapamiento y retorna true.

Test 04 - Conflicto por reunión completamente contenida:

```
@Test @elpeluca *
void testCompartimosHorarioContenidaDentroDeOtra () {
    agendaPersonal.anadirReunion(reunion1);
    Reunion reunion4 = new Reunion(LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 12, minute: 30),
        LocalDate.of( year: 2026, month: 6, dayOfMonth: 20, hour: 13, minute: 30), temaAtratar: "AS", lugarDeReunion: "av 123");
    assertTrue(agendaPersonal.existeConflictoReunion(reunion4));
}
```

Se registra una reunión en la agenda y se crea otra de forma local que comienza después y termina antes que la primera, quedando completamente dentro de su intervalo. El assertTrue verifica que existeConflicto detecta el conflicto y retorna true.

Test 05 - Parámetro nulo:

```
@Test @elpeluca
void testExisteConflictoReunionParametroNull () { assertTrue(agendaPersonal.existeConflictoReunion( nuevaReunion: null)); }
```

Se intenta invocar existeConflicto pasando null como argumento. El assertTrue verifica que el método detecta el valor inválido y lanza una excepción, confirmando que la contención funciona correctamente.

Funcionalidad - Método existeConflictoContacto:

```
public boolean existeConflictoContacto (Contacto nuevoContacto){ 5 usages  elpeluca
    if (nuevoContacto==null){
        return true;
    }
    int i = 0;
    boolean existe = false;
    while (i<this.contactos.size() && !existe){
        if (this.contactos.get(i).equals(nuevoContacto)){
            existe = true;
        }else {
            i++;
        }
    }
    return existe;
}
```

Descripción:

Este método recorre la lista de contactos de la agenda en busca de uno equivalente al que se recibe por parámetro. Si lo encuentra, retorna true. Incluye validación para manejar contactos null sin producir errores.

Test 01 - Contacto existente:

```
void testExisteConflictoIdentificarIgual () {
    agendaPersonal.anadirContacto(participante1);
    Contacto mismo = new Contacto( nombre: "Roberto", telefono: "2494066652", mail: "1@gmail");
    assertTrue(agendaPersonal.existeConflictoContacto(mismo));
}
```

Se añade un contacto a la agenda y se crea otro de forma local con el mismo nombre. El assertTrue verifica que existeConflicto los reconoce como equivalentes y retorna true.

Test 02 - Parámetro nulo:

```
@Test  elpeluca
void testExisteConflictoDatosNull () { assertTrue(agendaPersonal.existeConflictoContacto( nuevoContacto: null)); }
```

Se intenta invocar existeConflicto pasando null como argumento. El assertTrue verifica que el método detecta el valor inválido y lanza una excepción, confirmando que la contención ante entradas inválidas funciona correctamente.

Resultado de los test:

```
[INFO] Running AgendaPersonalTest  
Iniciando suite de pruebas de AgendaPersonal  
  
*  
*  
*  
*  
*  
*  
*  
*  
*  
*  
*  
*  
*  
*  
*  
*  
  
=== Suite finalizada ===  
[WARNING] Tests run: 15, Failures: 0, Errors: 0, Skipped: 1, Time elapsed: 0.070 s -- in AgendaPersonalTest
```

Todos los test pasaron

Reflexión final:

El testeo es una parte importante del desarrollo de software que nos permite detectar errores en etapa temprana así reduciendo los costos de producción y evitando percances en producción que puedan generar pérdidas. Fue muy interesante practicarlo pese a que no me he adaptado a las buenas prácticas.

Repositorio de git:

https://github.com/ElPelucaDLAP/TP1_JUnit_AgustinCarsaniga